Radiological Hazard Assessment Tool

Complete Code Documentation

Line-by-Line Explanation with Python Documentation Links

Hamphrey Tumusiime

June 25, 2026

Abstract

Hamphrey's documentation

Contents

1 Introduction

1.1 Document Overview

This document provides a complete line-by-line explanation of the Radiological Hazard Assessment Tool source code. The application is written in Python 3 using the PyQt5 framework for the graphical user interface, pandas for data manipulation, NumPy for numerical computations, and Matplotlib for data visualization.

1.2 Application Purpose

The tool computes five standard radiological hazard indices from measurements of naturally occurring radioactive materials (NORM):

- Radium Equivalent Activity (Rad_Eq)
- External Hazard Index (Hex)
- Internal Hazard Index (Hin)
- Absorbed Dose Rate (Dose_Rate)
- Annual Effective Dose Equivalent (AEDE)

1.3 Documentation Conventions

- Each code section is presented with line numbers
- Lines are explained with detailed descriptions
- Links to official Python documentation are provided for key functions
- Mathematical equations are presented in LaTeX format

1.4 Technology Stack

Table 1: Technology Stack with Documentation Links

Component	Version	Documentation
Python	3.7+	https://docs.python.org/3/
PyQt5	5.15+	https://doc.qt.io/qt-5/
pandas	1.3+	https://pandas.pydata.org/docs/
NumPy	1.21+	https://numpy.org/doc/stable/
Matplotlib	3.5+	https://matplotlib.org/stable/contents.html

2 Import Statements

```
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as
    FigureCanvas
from matplotlib.figure import Figure

from PyQt5.QtCore import Qt
from PyQt5.QtGui import QDoubleValidator
from PyQt5.QtWidgets import (
    QApplication, QWidget, QLabel, QPushButton,
    QVBoxLayout, QHBoxLayout,
    QFileDialog, QTableWidget, QTableWidgetItem,
    QGroupBox, QProgressBar, QSplitter,
    QHeaderView, QComboBox, QTabWidget
)
```

Listing 1: Import Statements

2.1 Line-by-Line Explanation

2.1.1 Line 1: `import sys`

- **Purpose**: Imports the Python system module
- **Documentation**: https://docs.python.org/3/library/sys.html
- **Usage in Code**:
  - `sys.argv`: Passes command-line arguments to QApplication
  - `sys.exit()`: Ensures proper program termination
- **Why needed**: Required for Qt application initialization and clean exit

2.1.2 Line 2: `import numpy as np`

- **Purpose**: Imports NumPy for numerical computing
- **Documentation**: https://numpy.org/doc/stable/reference/
- **Usage in Code**:
  - `np.sqrt()`: Square root for uncertainty propagation
  - `np.argsort()`: Sorting indices for clean plotting
  - `np.polyfit()`: Linear regression
  - `np.corrcoef()`: Correlation coefficient for R^2
  - `np.mean()`, `np.std()`: Statistical calculations
- **Alias**: `np` is the standard NumPy alias

2.1.3 Line 3: `import pandas as pd`

- **Purpose**: Imports pandas for data manipulation
- **Documentation**: https://pandas.pydata.org/docs/reference/index.html
- **Usage in Code**:
  - `pd.read_csv()`: Load CSV files
  - `pd.DataFrame()`: Create data structures
  - `DataFrame.to_csv()`: Export results
  - `DataFrame.apply()`: Apply functions to rows/columns
  - `Series.round()`: Round numeric values
- **Alias**: `pd` is the standard pandas alias

2.1.4 Line 4: `import matplotlib.pyplot as plt`

- **Purpose**: Imports Matplotlib's pyplot interface
- **Documentation**: https://matplotlib.org/stable/api/pyplot_summary.html
- **Usage in Code**: Used for plot creation and customization
- **Note**: Mainly used indirectly through the object-oriented API

2.1.5 Line 5: `from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas`

- **Purpose**: Imports the Qt5 backend for embedding Matplotlib in PyQt5
- **Documentation**: https://matplotlib.org/stable/api/backend_qt5agg_api.html
- **What it does**:
  - Wraps a Matplotlib Figure as a Qt widget
  - Allows plots to be displayed inside the GUI
- **Alias**: Renamed to `FigureCanvas` for convenience

2.1.6 Line 6: `from matplotlib.figure import Figure`

- **Purpose**: Imports the Matplotlib Figure class
- **Documentation**: https://matplotlib.org/stable/api/figure_api.html
- **Usage**: Creates the canvas for plotting
- **Key Methods**:
  - `Figure.clear()`: Clear the figure
  - `Figure.add_subplot()`: Add axes
  - `Figure.savefig()`: Save to file
  - `Figure.tight_layout()`: Adjust spacing

2.1.7 Line 8: `from PyQt5.QtCore import Qt`

- **Purpose**: Imports Qt namespace constants
- **Documentation**: https://doc.qt.io/qt-5/qt.html
- **Usage in Code**:
  - `Qt.green`, `Qt.red`: Color constants for table cells
  - `Qt.Horizontal`: Splitter orientation

2.1.8 Line 9: `from PyQt5.QtGui import QDoubleValidator`

- **Purpose**: Imports validator for numeric input
- **Documentation**: https://doc.qt.io/qt-5/qdoublevalidator.html
- **Usage**: Restricts QLineEdit to valid floating-point numbers
- **Why**: Prevents non-numeric input in manual entry fields

2.1.9 Lines 10-18: `from PyQt5.QtWidgets import ...`

- **Purpose**: Imports all Qt widget classes used in the GUI
- **Documentation**: https://doc.qt.io/qt-5/qtwidgets-module.html
- **Widgets Imported**:
  - `QApplication`: Application object (must be created first)
  - `QWidget`: Base class for all UI elements
  - `QLabel`: Displays text
  - `QPushButton`: Clickable button
  - `QVBoxLayout`, `QHBoxLayout`: Layout managers
  - `QFileDialog`: File open/save dialog
  - `QTableWidget`, `QTableWidgetItem`: Table display
  - `QGroupBox`: Grouped container with border
  - `QProgressBar`: Progress indicator
  - `QSplitter`: Resizable panel divider
  - `QHeaderView`: Table header
  - `QComboBox`: Dropdown selection
  - `QTabWidget`: Tabbed interface

3 Computation Engine

3.1 Function: compute_indices()

```python
def compute_indices(df):
    """
    Compute hazard indices with uncertainty propagation.
    Returns DataFrame with value and uncertainty in separate columns.
    """
    df = df.copy()

    # Extract uncertainties (default 0 if not present)
    dRa = df.get("dRa", pd.Series([0] * len(df)))
    dTh = df.get("dTh", pd.Series([0] * len(df)))
    dK = df.get("dK", pd.Series([0] * len(df)))

    # Calculate values
    rad_eq = df["Ra"] + 1.43 * df["Th"] + 0.077 * df["K"]
    hex_val = df["Ra"]/370 + df["Th"]/259 + df["K"]/4810
    hin_val = df["Ra"]/185 + df["Th"]/259 + df["K"]/4810
    dose_rate = 0.462*df["Ra"] + 0.604*df["Th"] + 0.0417*df["K"]
    aede = dose_rate * 8760 * 0.2 * 1e-6

    # Propagate uncertainties
    dRadEq = np.sqrt((1*dRa)**2 + (1.43*dTh)**2 + (0.077*dK)**2)
    dHex = np.sqrt((dRa/370)**2 + (dTh/259)**2 + (dK/4810)**2)
    dHin = np.sqrt((dRa/185)**2 + (dTh/259)**2 + (dK/4810)**2)
    dDose = np.sqrt((0.462*dRa)**2 + (0.604*dTh)**2 + (0.0417*dK)**2)
    dAEDE = dDose * 8760 * 0.2 * 1e-6

    return pd.DataFrame({
        "Rad_Eq": rad_eq, "dRad_Eq": dRadEq,
        "Hex": hex_val, "dHex": dHex,
        "Hin": hin_val, "dHin": dHin,
        "Dose_Rate": dose_rate, "dDose_Rate": dDose,
        "AEDE": aede, "dAEDE": dAEDE
    })
```

Listing 2: Computation Engine

3.2 Line-by-Line Explanation

3.2.1 Line 1: def compute_indices(df):

- **Purpose**: Defines the main computation function
- **Parameter**: df - pandas DataFrame with columns 'Ra', 'Th', 'K', 'dRa', 'dTh', 'dK'
- **Returns**: pandas DataFrame with computed indices and uncertainties
- **Documentation**: https://docs.python.org/3/tutorial/controlflow.html#defining-functions

3.2.2 Line 2-4: Docstring

- **Purpose**: Documents the function's purpose
- **Content**: Explains what the function does and what it returns
- **Best Practice**: Always include docstrings for functions

3.2.3 Line 5: `df = df.copy()`

- **Purpose**: Creates a copy of the DataFrame
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.copy.html
- **Why**: Prevents modifying the original DataFrame (defensive programming)
- **Consequence**: Changes inside function don't affect caller's data

3.2.4 Line 8-10: Extract Uncertainties

```
8  dRa = df.get("dRa", pd.Series([0] * len(df)))
9  dTh = df.get("dTh", pd.Series([0] * len(df)))
10 dK = df.get("dK", pd.Series([0] * len(df)))
```

- **Method**: `DataFrame.get()`
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.get.html
- **What it does**:
  - If column exists → returns it
  - If column doesn't exist → returns default value (Series of zeros)
- **Why**: Allows CSV files without uncertainty columns
- **Default**: `pd.Series([0] * len(df))` creates zeros for each row

3.2.5 Line 13-17: Calculate Hazard Indices

```
13 rad_eq = df["Ra"] + 1.43 * df["Th"] + 0.077 * df["K"]
14 hex_val = df["Ra"]/370 + df["Th"]/259 + df["K"]/4810
15 hin_val = df["Ra"]/185 + df["Th"]/259 + df["K"]/4810
16 dose_rate = 0.462*df["Ra"] + 0.604*df["Th"] + 0.0417*df["K"]
17 aede = dose_rate * 8760 * 0.2 * 1e-6
```

Line 13: `rad_eq = df["Ra"] + 1.43 * df["Th"] + 0.077 * df["K"]`

- **Formula**: Radium Equivalent Activity
- **Equation**: $Ra_{eq} = C_{Ra} + 1.43C_{Th} + 0.077C_K$
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.Series.html
- **Operation**: Vectorized arithmetic (applies to all rows simultaneously)
- **Unit**: Bq/kg

Line 14: `hex_val = df["Ra"]/370 + df["Th"]/259 + df["K"]/4810`

- **Formula**: External Hazard Index
- **Equation**: $H_{ex} = \frac{C_{Ra}}{370} + \frac{C_{Th}}{259} + \frac{C_K}{4810}$
- **Threshold**: 1
- **Unit**: Dimensionless

Line 15: `hin_val = df["Ra"]/185 + df["Th"]/259 + df["K"]/4810`

- **Formula**: Internal Hazard Index
- **Equation**: $H_{in} = \frac{C_{Ra}}{185} + \frac{C_{Th}}{259} + \frac{C_K}{4810}$
- **Threshold**: 1
- **Unit**: Dimensionless

Line 16: `dose_rate = 0.462*df["Ra"] + 0.604*df["Th"] + 0.0417*df["K"]`

- **Formula**: Absorbed Dose Rate
- **Equation**: $\dot{D} = 0.462C_{Ra} + 0.604C_{Th} + 0.0417C_K$
- **Threshold**: 59 nGy/h
- **Unit**: nGy/h

Line 17: `aede = dose_rate * 8760 * 0.2 * 1e-6`

- **Formula**: Annual Effective Dose Equivalent
- **Equation**: $AEDE = \dot{D} \times 8760 \times 0.2 \times 10^{-6}$
- **Constants**:
  - 8760 = hours in a year
  - 0.2 = outdoor occupancy factor
  - 1e-6 = nGy to mGy conversion
- **Threshold**: 1 mSv/y
- **Unit**: mSv/y

3.2.6 Line 20-24: Propagate Uncertainties

```
20 dRadEq = np.sqrt((1*dRa)**2 + (1.43*dTh)**2 + (0.077*dK)**2)
21 dHex = np.sqrt((dRa/370)**2 + (dTh/259)**2 + (dK/4810)**2)
22 dHin = np.sqrt((dRa/185)**2 + (dTh/259)**2 + (dK/4810)**2)
23 dDose = np.sqrt((0.462*dRa)**2 + (0.604*dTh)**2 + (0.0417*dK)**2)
24 dAEDE = dDose * 8760 * 0.2 * 1e-6
```

Line 20: `dRadEq = np.sqrt((1*dRa)**2 + (1.43*dTh)**2 + (0.077*dK)**2)`

- **Method**: `np.sqrt()`
- **Documentation**: https://numpy.org/doc/stable/reference/generated/numpy.sqrt.html
- **Formula**: $\sigma_{Ra_{eq}} = \sqrt{(1 \cdot \sigma_{Ra})^2 + (1.43 \cdot \sigma_{Th})^2 + (0.077 \cdot \sigma_K)^2}$
- **Principle**: Error propagation for independent variables
- **Operation**: Element-wise square root on Series

Line 21: `dHex = np.sqrt((dRa/370)**2 + (dTh/259)**2 + (dK/4810)**2)`

- **Formula**: $\sigma_{H_{ex}} = \sqrt{(\frac{\sigma_{Ra}}{370})^2 + (\frac{\sigma_{Th}}{259})^2 + (\frac{\sigma_K}{4810})^2}$
- **Method**: `np.sqrt()` for element-wise square root

Line 22: `dHin = np.sqrt((dRa/185)**2 + (dTh/259)**2 + (dK/4810)**2)`

- **Formula**: $\sigma_{H_{in}} = \sqrt{(\frac{\sigma_{Ra}}{185})^2 + (\frac{\sigma_{Th}}{259})^2 + (\frac{\sigma_K}{4810})^2}$

Line 23: `dDose = np.sqrt((0.462*dRa)**2 + (0.604*dTh)**2 + (0.0417*dK)**2)`

- **Formula**: $\sigma_{\dot{D}} = \sqrt{(0.462 \cdot \sigma_{Ra})^2 + (0.604 \cdot \sigma_{Th})^2 + (0.0417 \cdot \sigma_K)^2}$

Line 24: `dAEDE = dDose * 8760 * 0.2 * 1e-6`

- **Formula**: $\sigma_{AEDE} = \sigma_{\dot{D}} \times 8760 \times 0.2 \times 10^{-6}$
- **Note**: Scalar multiplication (no square root needed)

3.2.7 Line 26-33: Return DataFrame

```
26 return pd.DataFrame({
27     "Rad_Eq": rad_eq, "dRad_Eq": dRadEq,
28     "Hex": hex_val, "dHex": dHex,
29     "Hin": hin_val, "dHin": dHin,
30     "Dose_Rate": dose_rate, "dDose_Rate": dDose,
31     "AEDE": aede, "dAEDE": dAEDE
32 })
```

- **Method**: `pd.DataFrame()`

- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html
- **Structure**: Interleaved value and uncertainty columns
- **Column Naming**: ‘”Value”‘ and ‘”dValue”‘ for each index
- **Why Interleaved**: Easy to read and compare values with uncertainties

4 Main Application Class

4.1 Class Declaration and Initialization

```
class MainApp(QWidget):

    def __init__(self):
        super().__init__()

        self.setWindowTitle("Radiological Hazard Assessment Tool")
        self.resize(1400, 800)

        self.data = None
        self.results = None

        # Main layout
        self.layout = QHBoxLayout()
        self.setLayout(self.layout)
```

Listing 3: MainApp Class Initialization

4.2 Line-by-Line Explanation

4.2.1 Line 1: `class MainApp(QWidget):`

- **Purpose**: Defines the main application class
- **Inheritance**: Inherits from `QWidget`
- **Documentation**: https://doc.qt.io/qt-5/qwidget.html
- **Why QWidget**: Base class for all PyQt5 GUI elements
- **Inheritance Benefits**:
  - Window management
  - Event handling
  - Layout support
  - Styling capabilities

4.2.2 Line 3: `def __init__(self):`

- **Purpose**: Constructor method
- **Documentation**: https://docs.python.org/3/reference/datamodel.html#object.__init__
- **Called**: Automatically when `MainApp()` is created
- **Self**: Reference to the instance being created

4.2.3 Line 4: `super().__init__()`

- **Purpose**: Calls the parent class (QWidget) constructor
- **Documentation**: https://docs.python.org/3/library/functions.html#super
- **Why Critical**: Initializes the QWidget part of the object
- **Without this**: The widget wouldn't be properly initialized (crash)

4.2.4 Line 6: `self.setWindowTitle("Radiological Hazard Assessment Tool")`

- **Method**: `QWidget.setWindowTitle()`
- **Documentation**: https://doc.qt.io/qt-5/qwidget.html#windowTitle-prop
- **Purpose**: Sets the text in the window title bar
- **Result**: Title bar shows "Radiological Hazard Assessment Tool"

4.2.5 Line 7: `self.resize(1400, 800)`

- **Method**: `QWidget.resize()`
- **Documentation**: https://doc.qt.io/qt-5/qwidget.html#resize
- **Parameters**: Width (1400px), Height (800px)
- **Purpose**: Sets initial window size
- **User Can**: Still resize the window

4.2.6 Line 9-10: Instance Variables

```
9  self.data = None
10 self.results = None
```

- **Purpose**: Store application data
- `self.data`: Raw CSV data (pandas DataFrame)
- `self.results`: Computed results (pandas DataFrame)
- **Initial Value**: None (no data loaded yet)
- **Why Instance Variables**: Accessible from all methods

4.2.7 Line 13-14: Main Layout

```
13 self.layout = QHBoxLayout()
14 self.setLayout(self.layout)
```

- **Method**: `QHBoxLayout()`
- **Documentation**: https://doc.qt.io/qt-5/qhboxlayout.html
- **Purpose**: Horizontal layout (left to right)
- **Structure**: Left panel (controls) — Right panel (results)
- **setLayout()**: Attaches layout to window

4.3 Left Panel Construction

```
# ------------------------------------------
# LEFT PANEL - CONTROLS
# ------------------------------------------
left_panel = QWidget()
left_panel.setFixedWidth(350)
left_layout = QVBoxLayout(left_panel)

# Title
title = QLabel("Batch Processing")
title.setObjectName("title")
left_layout.addWidget(title)
```

Listing 4: Left Panel Construction

4.3.1 Line 17-19: Create Left Panel

```
left_panel = QWidget()
left_panel.setFixedWidth(350)
left_layout = QVBoxLayout(left_panel)
```

- **Line 17**: Creates container widget for left side
- **Line 18**: `setFixedWidth(350)` - locks width to 350px
- **Line 19**: `QVBoxLayout()` - vertical layout inside panel
- **Why Fixed Width**: Consistent control panel size
- **Documentation**: https://doc.qt.io/qt-5/qwidget.html#setFixedWidth

4.3.2 Line 22-24: Add Title

```
title = QLabel("Batch Processing")
title.setObjectName("title")
left_layout.addWidget(title)
```

- **Line 22**: Creates label with text "Batch Processing"
- **Line 23**: `setObjectName("title")` - for CSS styling
- **Line 24**: `addWidget()` - adds to layout
- **Documentation**: https://doc.qt.io/qt-5/qlabel.html

4.4 Data Input Group

```
# ------------------------------------------
# DATA INPUT
# ------------------------------------------
input_group = QGroupBox("Data Input")
input_layout = QVBoxLayout()
input_group.setLayout(input_layout)

file_layout = QHBoxLayout()
self.load_btn = QPushButton("       Load CSV")
self.load_btn.clicked.connect(self.load_csv)
self.file_label = QLabel("No file loaded")
self.file_label.setWordWrap(True)
file_layout.addWidget(self.load_btn)
file_layout.addWidget(self.file_label)
input_layout.addLayout(file_layout)

self.sample_info_label = QLabel("Samples: 0")
input_layout.addWidget(self.sample_info_label)

left_layout.addWidget(input_group)
```

Listing 5: Data Input Group

4.4.1 Line 28-30: Create Group Box

```
input_group = QGroupBox("Data Input")
input_layout = QVBoxLayout()
input_group.setLayout(input_layout)
```

- **Line 28**: `QGroupBox("Data Input")` - bordered box with title
- **Line 29**: Creates vertical layout inside the group
- **Line 30**: Attaches layout to the group
- **Documentation**: https://doc.qt.io/qt-5/qgroupbox.html

4.4.2 Line 32-40: File Loading Row

```
file_layout = QHBoxLayout()
self.load_btn = QPushButton("       Load CSV")
self.load_btn.clicked.connect(self.load_csv)
self.file_label = QLabel("No file loaded")
self.file_label.setWordWrap(True)
file_layout.addWidget(self.load_btn)
file_layout.addWidget(self.file_label)
input_layout.addLayout(file_layout)
```

- **Line 32**: Horizontal layout for button and label
- **Line 33**: Creates load button with emoji
- **Line 34**: `clicked.connect(self.load_csv)` - signal-slot connection

- **Line 35**: Label showing loaded file status
- **Line 36**: `setWordWrap(True)` - wraps long filenames
- **Line 37-38**: Adds widgets to horizontal layout
- **Line 39**: Adds horizontal row to vertical layout
- **Signal-Slot Documentation**: https://doc.qt.io/qt-5/signalsandslots.html

4.4.3 Line 42-43: Sample Info

```
42 self.sample_info_label = QLabel("Samples: 0")
43 input_layout.addWidget(self.sample_info_label)
```

- Creates label showing sample count
- Initially "Samples: 0"
- Updated when CSV is loaded

4.4.4 Line 45: Add Group to Layout

```
45 left_layout.addWidget(input_group)
```

- Adds the entire Data Input group to left panel
- The group appears as a bordered box with all its contents

4.5 CSV Format Info Group

```
# ------------------------------------------------
# CSV FORMAT INFO
# ------------------------------------------------
info_group = QGroupBox("CSV Format")
info_layout = QVBoxLayout()
info_group.setLayout(info_layout)

info_text = QLabel(
    "Required columns:\n"
    "Ra, Th, K (activity concentrations)\n"
    "dRa, dTh, dK (uncertainties)\n\n"
    "If uncertainties are not provided,\n"
    "they will be set to zero."
)
info_text.setWordWrap(True)
info_layout.addWidget(info_text)

left_layout.addWidget(info_group)
```

Listing 6: CSV Format Info Group

4.5.1 Line 48-50: Create Group Box

- `QGroupBox("CSV Format")`: Creates bordered box
- `QVBoxLayout()`: Vertical layout inside
- `setLayout()`: Attaches layout

4.5.2 Line 52-60: Info Text

```
info_text = QLabel(
    "Required columns:\n"
    "Ra, Th, K (activity concentrations)\n"
    "dRa, dTh, dK (uncertainties)\n\n"
    "If uncertainties are not provided,\n"
    "they will be set to zero."
)
```

- `QLabel()`: Creates text label
- `\n`: Newline characters for multi-line text
- **Content**: Explains required CSV format
- **Important**: Informs users about uncertainty handling

4.5.3 Line 61-63

- `setWordWrap(True)`: Wraps text if too wide
- `addWidget(info_text)`: Adds to layout
- `addWidget(info_group)`: Adds group to left panel

4.6 Actions Group

```
# ---------------------------------------------
# ACTIONS
# ---------------------------------------------
action_group = QGroupBox("Actions")
action_layout = QVBoxLayout()
action_group.setLayout(action_layout)

self.process_btn = QPushButton("    Process Samples")
self.process_btn.clicked.connect(self.compute_all)
self.process_btn.setEnabled(False)
action_layout.addWidget(self.process_btn)

self.progress_bar = QProgressBar()
action_layout.addWidget(self.progress_bar)

# Export buttons
export_layout = QHBoxLayout()
self.export_btn = QPushButton("       Export CSV")
self.export_btn.clicked.connect(self.save_results)
self.export_btn.setEnabled(False)

self.save_plot_btn = QPushButton("       Save Plot")
self.save_plot_btn.clicked.connect(self.save_plot)
self.save_plot_btn.setEnabled(False)

export_layout.addWidget(self.export_btn)
export_layout.addWidget(self.save_plot_btn)
action_layout.addLayout(export_layout)

left_layout.addWidget(action_group)
```

Listing 7: Actions Group

4.6.1 Line 68-70: Create Actions Group

- `QGroupBox("Actions")`: Group for action buttons
- `QVBoxLayout()`: Vertical layout
- Groups all action-related widgets together

4.6.2 Line 72-75: Process Button

```
self.process_btn = QPushButton("    Process Samples")
self.process_btn.clicked.connect(self.compute_all)
self.process_btn.setEnabled(False)
action_layout.addWidget(self.process_btn)
```

- **Line 72**: Creates process button with lightning emoji
- **Line 73**: Connects to `compute_all()` method
- **Line 74**: `setEnabled(False)` - initially disabled

- **Why Disabled**: No data loaded yet
- **Documentation**: https://doc.qt.io/qt-5/qpushbutton.html

4.6.3 Line 77-78: Progress Bar

```
77 self.progress_bar = QProgressBar()
78 action_layout.addWidget(self.progress_bar)
```

- `QProgressBar()`: Shows processing progress
- Initially empty (0
- Updated during computation
- **Documentation**: https://doc.qt.io/qt-5/qprogressbar.html

4.6.4 Line 81-91: Export Buttons

```
81 export_layout = QHBoxLayout()
82 self.export_btn = QPushButton("       Export CSV")
83 self.export_btn.clicked.connect(self.save_results)
84 self.export_btn.setEnabled(False)
85
86 self.save_plot_btn = QPushButton("       Save Plot")
87 self.save_plot_btn.clicked.connect(self.save_plot)
88 self.save_plot_btn.setEnabled(False)
89
90 export_layout.addWidget(self.export_btn)
91 export_layout.addWidget(self.save_plot_btn)
92 action_layout.addLayout(export_layout)
```

- **Line 81**: Horizontal layout for two buttons
- **Line 82-84**: Export CSV button (initially disabled)
- **Line 86-88**: Save Plot button (initially disabled)
- **Line 90-91**: Add buttons to horizontal layout
- **Line 92**: Add horizontal row to vertical layout

4.6.5 Line 94: Add to Layout

```
94 left_layout.addWidget(action_group)
```

- Adds Actions group to left panel

4.7 Plot Options Group

```
# ------------------------------------------
# PLOT OPTIONS
# ------------------------------------------
plot_group = QGroupBox("Plot Options")
plot_layout = QVBoxLayout()
plot_group.setLayout(plot_layout)

# Plot view selector
view_layout = QHBoxLayout()
view_layout.addWidget(QLabel("View:"))
self.plot_view = QComboBox()
self.plot_view.addItems([
    "Combined (Ra + Th + K)",
    "Ra Only",
    "Th Only",
    "K Only"
])
self.plot_view.currentTextChanged.connect(self.update_plot)
view_layout.addWidget(self.plot_view)
plot_layout.addLayout(view_layout)

self.plot_btn = QPushButton("       Generate Plot")
self.plot_btn.clicked.connect(self.update_plot)
self.plot_btn.setEnabled(False)
plot_layout.addWidget(self.plot_btn)

left_layout.addWidget(plot_group)
```

Listing 8: Plot Options Group

4.7.1 Line 99-101: Create Plot Options Group

- `QGroupBox("Plot Options")`: Group for plot controls
- `QVBoxLayout()`: Vertical layout

4.7.2 Line 104-115: View Selector

```
view_layout = QHBoxLayout()
view_layout.addWidget(QLabel("View:"))
self.plot_view = QComboBox()
self.plot_view.addItems([
    "Combined (Ra + Th + K)",
    "Ra Only",
    "Th Only",
    "K Only"
])
self.plot_view.currentTextChanged.connect(self.update_plot)
view_layout.addWidget(self.plot_view)
plot_layout.addLayout(view_layout)
```

- **Line 104**: Horizontal layout for label + dropdown
- **Line 105**: Adds "View:" label

- **Line 106**: `QComboBox()` - dropdown selection
- **Line 107-111**: Adds four viewing options
- **Line 112**: `currentTextChanged.connect()` - auto-updates plot
- **Line 113**: Adds dropdown to horizontal layout
- **Line 114**: Adds row to vertical layout
- **Documentation**: https://doc.qt.io/qt-5/qcombobox.html

4.7.3 Line 117-120: Generate Plot Button

```
117 self.plot_btn = QPushButton("       Generate Plot")
118 self.plot_btn.clicked.connect(self.update_plot)
119 self.plot_btn.setEnabled(False)
120 plot_layout.addWidget(self.plot_btn)
```

- Creates manual plot refresh button
- Initially disabled (no data processed)
- Same function as dropdown (updates plot)

4.7.4 Line 122: Add to Layout

```
122 left_layout.addWidget(plot_group)
```

4.8 Safety Thresholds Group

```
# ---------------------------------------------
# THRESHOLD INFORMATION
# ---------------------------------------------
info_group = QGroupBox("Safety Thresholds")
info_layout = QVBoxLayout()
info_group.setLayout(info_layout)

thresholds_text = QLabel(
    "Radium Equivalent:     370 Bq/kg\n"
    "External Hazard (Hex):     1\n"
    "Internal Hazard (Hin):     1\n"
    "Dose Rate:     59 nGy/h\n"
    "Annual Effective Dose:     1 mSv/y"
)
thresholds_text.setWordWrap(True)
info_layout.addWidget(thresholds_text)

left_layout.addWidget(info_group)
```

Listing 9: Safety Thresholds Group

4.8.1 Line 127-129: Create Group Box

- `QGroupBox("Safety Thresholds")`: Group for threshold info
- Shows all safety limits used for classification

4.8.2 Line 131-138: Threshold Text

```
thresholds_text = QLabel(
    "Radium Equivalent:     370 Bq/kg\n"
    "External Hazard (Hex):     1\n"
    "Internal Hazard (Hin):     1\n"
    "Dose Rate:     59 nGy/h\n"
    "Annual Effective Dose:     1 mSv/y"
)
```

- Lists all five safety thresholds
- Reference for user during analysis
- Based on international standards

4.8.3 Line 139-141

- `setWordWrap(True)`: Enables text wrapping
- `addWidget(thresholds_text)`: Adds to layout
- `addWidget(info_group)`: Adds group to left panel

4.9 Status Bar and Stretch

```
# Status bar at bottom of left panel
self.status = QLabel("Ready - Load CSV data to begin")
self.status.setObjectName("status")
left_layout.addWidget(self.status)

left_layout.addStretch()
```

Listing 10: Status Bar and Stretch

4.9.1 Line 144-147: Status Bar

```
144 self.status = QLabel("Ready - Load CSV data to begin")
145 self.status.setObjectName("status")
146 left_layout.addWidget(self.status)
```

- **Line 144**: Creates status label with initial message
- **Line 145**: `setObjectName("status")` for CSS styling
- **Line 146**: Adds to layout
- **Purpose**: Shows status messages to user
- **Updated**: Throughout application (loading, processing, saving)

4.9.2 Line 148: Add Stretch

```
148 left_layout.addStretch()
```

- **Method**: `QBoxLayout.addStretch()`
- **Documentation**: https://doc.qt.io/qt-5/qboxlayout.html#addStretch
- **Purpose**: Pushes all content to the top
- **Effect**: Empty space fills bottom of left panel
- **Without it**: Widgets would stretch to fill space

4.10 Right Panel Construction

```
# ------------------------------------------
# RIGHT PANEL - RESULTS WITH TABS
# ------------------------------------------
right_panel = QWidget()
right_layout = QVBoxLayout(right_panel)

# Create tab widget for plot and table
self.tabs = QTabWidget()

# ----- PLOT TAB -----
plot_tab = QWidget()
plot_layout = QVBoxLayout(plot_tab)

self.figure = Figure(figsize=(10, 7))
self.canvas = FigureCanvas(self.figure)
plot_layout.addWidget(self.canvas)

self.tabs.addTab(plot_tab, "       Correlation Plot")

# ----- TABLE TAB -----
table_tab = QWidget()
table_layout = QVBoxLayout(table_tab)

self.table = QTableWidget()
self.table.horizontalHeader().setSectionResizeMode(QHeaderView.
    ResizeToContents)
table_layout.addWidget(self.table)

self.tabs.addTab(table_tab, "       Results Table")

right_layout.addWidget(self.tabs)

# Set splitter sizes
self.layout.addWidget(left_panel)
self.layout.addWidget(right_panel)

# Apply styles
self.apply_style()
```

Listing 11: Right Panel Construction

4.10.1 Line 152-154: Create Right Panel

```
right_panel = QWidget()
right_layout = QVBoxLayout(right_panel)
```

- Creates container for right side
- Vertical layout for tab widget
- Takes remaining space (left panel is fixed width)

4.10.2 Line 157: Create Tab Widget

```
157 self.tabs = QTabWidget()
```

- **Method**: `QTabWidget()`
- **Documentation**: https://doc.qt.io/qt-5/qtabwidget.html
- **Purpose**: Tabbed interface for plot and table
- **Benefits**: Clean separation of views

4.10.3 Line 160-168: Plot Tab

```
160 plot_tab = QWidget()
161 plot_layout = QVBoxLayout(plot_tab)
162
163 self.figure = Figure(figsize=(10, 7))
164 self.canvas = FigureCanvas(self.figure)
165 plot_layout.addWidget(self.canvas)
166
167 self.tabs.addTab(plot_tab, "       Correlation Plot")
```

- **Line 160-161**: Creates container and layout for plot
- **Line 163**: `Figure(figsize=(10, 7))` - Matplotlib figure
- **Line 164**: `FigureCanvas()` - Qt wrapper for figure
- **Line 165**: Adds canvas to layout
- **Line 167**: Adds tab with ” Correlation Plot” label

4.10.4 Line 170-179: Table Tab

```
170 table_tab = QWidget()
171 table_layout = QVBoxLayout(table_tab)
172
173 self.table = QTableWidget()
174 self.table.horizontalHeader().setSectionResizeMode(QHeaderView.
    ResizeToContents)
175 table_layout.addWidget(self.table)
176
177 self.tabs.addTab(table_tab, "       Results Table")
```

- **Line 170-171**: Creates container and layout for table
- **Line 173**: `QTableWidget()` - spreadsheet-style table
- **Line 174**: Auto-resize columns to fit content
- **Line 175**: Adds table to layout
- **Line 177**: Adds tab with ” Results Table” label

4.10.5 Line 181-185: Final Assembly

```
181 right_layout.addWidget(self.tabs)
182
183 # Set splitter sizes
184 self.layout.addWidget(left_panel)
185 self.layout.addWidget(right_panel)
186
187 # Apply styles
188 self.apply_style()
```

- **Line 181**: Adds tabs to right panel
- **Line 184**: Adds left panel to main layout
- **Line 185**: Adds right panel to main layout
- **Line 188**: Applies CSS styles

5 Data Loading Method

```
def load_csv(self):

    path, _ = QFileDialog.getOpenFileName(
        self,
        "Select CSV File",
        "",
        "CSV Files (*.csv)"
    )

    if not path:
        return

    try:
        df = pd.read_csv(path)

        required = ["Ra", "Th", "K", "dRa", "dTh", "dK"]

        # Check for required columns, add zeros if missing
        missing = []
        for col in required:
            if col not in df.columns:
                df[col] = 0.0
                missing.append(col)

        if missing:
            self.status.setText(f"       Columns not found, set to zero
    : {', '.join(missing)}")
            self.status.setStyleSheet("color: orange;")
        else:
            self.status.setText(f"    Loaded {len(df)} samples")
            self.status.setStyleSheet("color: lightgreen;")

        self.data = df[required]
        self.file_label.setText(f"Loaded: {path.split('/')[-1]}")
        self.sample_info_label.setText(f"Samples: {len(self.data)}")
        self.process_btn.setEnabled(True)

    except Exception as e:
        self.status.setText("    Error loading CSV")
        self.status.setStyleSheet("color: red;")
        print(e)
```

Listing 12: CSV Loading Method

5.1 Line-by-Line Explanation

5.1.1 Line 1: `def load_csv(self):`

- Defines the CSV loading method
- Connected to "Load CSV" button click signal

5.1.2 Line 3-8: File Dialog

```
path, _ = QFileDialog.getOpenFileName(
    self,
    "Select CSV File",
    "",
    "CSV Files (*.csv)"
)
```

- **Method**: `QFileDialog.getOpenFileName()`
- **Documentation**: https://doc.qt.io/qt-5/qfiledialog.html#getOpenFileName
- **Parameters**:
  - `self`: Parent widget
  - `"Select CSV File"`: Dialog title
  - `""`: Starting directory (current)
  - `"CSV Files (*.csv)"`: File filter
- **Returns**: `(path, filter)` - path is file path, `_` ignores filter

5.1.3 Line 10-11: Check Cancel

```
if not path:
    return
```

- If user cancels, `path` is empty string
- Returns without doing anything

5.1.4 Line 13-14: Read CSV

```
try:
    df = pd.read_csv(path)
```

- `try`: Starts error handling block
- `pd.read_csv(path)`: Reads CSV into DataFrame
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
- **If fails**: Jumps to `except` block

5.1.5 Line 16: Required Columns

```
required = ["Ra", "Th", "K", "dRa", "dTh", "dK"]
```

- List of required column names
- All six columns needed for computation

5.1.6 Line 19-24: Check Missing Columns

```
missing = []
for col in required:
    if col not in df.columns:
        df[col] = 0.0
        missing.append(col)
```

- `missing = []`: Empty list for missing columns
- Loops through required columns
- If column missing: adds with zeros, records in missing list
- **Why**: Allows CSV without uncertainty columns

5.1.7 Line 26-31: Status Messages

```
if missing:
    self.status.setText(f"       Columns not found, set to zero: {', '.
    join(missing)}")
    self.status.setStyleSheet("color: orange;")
else:
    self.status.setText(f"    Loaded {len(df)} samples")
    self.status.setStyleSheet("color: lightgreen;")
```

- **If missing**: Warning message with column names
- **Color**: Orange (warning)
- **Else**: Success message with sample count
- **Color**: Light green (success)

5.1.8 Line 33-36: Update UI

```
self.data = df[required]
self.file_label.setText(f"Loaded: {path.split('/')[-1]}")
self.sample_info_label.setText(f"Samples: {len(self.data)}")
self.process_btn.setEnabled(True)
```

- `self.data = df[required]`: Stores only required columns
- `path.split('/')[-1]`: Extracts filename from full path
- Updates file label with filename
- Updates sample count
- Enables Process button (was disabled)

5.1.9 Line 38-41: Error Handling

```
38 except Exception as e:
39     self.status.setText("    Error loading CSV")
40     self.status.setStyleSheet("color: red;")
41     print(e)
```

- Catches ANY exception during loading
- Shows red error message
- Prints details to console (for debugging)
- Application doesn't crash

6 Computation Method

```
def compute_all(self):

    if self.data is None:
        self.status.setText("No data loaded")
        return

    try:
        self.process_btn.setEnabled(False)
        self.progress_bar.setValue(0)

        df = self.data.copy()

        df["Sample"] = [
            f"SAMPLE_{i+1:03d}"
            for i in range(len(df))
        ]

        indices = compute_indices(df[["Ra", "Th", "K", "dRa", "dTh", "
    dK"]])

        # Round values (AEDE gets 6 dp, others 3 dp)
        for col in indices.columns:
            if "AEDE" in col:
                indices[col] = indices[col].round(6)
            else:
                indices[col] = indices[col].round(3)

        indices.insert(0, "Sample", df["Sample"])

        indices["Status"] = indices.apply(
            lambda row:
            "Safe"
            if (
                row["Rad_Eq"] <= 370 and
                row["Hex"] <= 1 and
                row["Hin"] <= 1 and
                row["Dose_Rate"] <= 59 and
                row["AEDE"] <= 1
            )
            else "Unsafe",
            axis=1
        )

        self.results = indices

        # ----------------------------------------
        # TABLE DISPLAY
        # ----------------------------------------
        self.table.setRowCount(len(indices))
        self.table.setColumnCount(len(indices.columns))
        self.table.setHorizontalHeaderLabels(indices.columns)

        for i in range(len(indices)):
            for j in range(len(indices.columns)):

```

```
                item = QTableWidgetItem(str(indices.iloc[i, j]))

                if indices.columns[j] == "Status":
                    if indices.iloc[i, j] == "Safe":
                        item.setBackground(Qt.green)
                    else:
                        item.setBackground(Qt.red)

                self.table.setItem(i, j, item)

        # Enable buttons
        self.export_btn.setEnabled(True)
        self.save_plot_btn.setEnabled(True)
        self.plot_btn.setEnabled(True)
        self.process_btn.setEnabled(True)
        self.progress_bar.setValue(100)

        safe_count = (indices["Status"] == "Safe").sum()
        unsafe_count = (indices["Status"] == "Unsafe").sum()

        self.status.setText(
            f"    Processing complete! {safe_count} safe, {unsafe_count} unsafe samples"
        )
        self.status.setStyleSheet("color: lightgreen;")

        # Switch to table tab to show results
        self.tabs.setCurrentIndex(1)

        # Generate initial plot
        self.update_plot()

    except Exception as e:
        self.status.setText("    Computation Error")
        self.status.setStyleSheet("color: red;")
        self.process_btn.setEnabled(True)
        print(e)
```

Listing 13: Computation Method

6.1 Line-by-Line Explanation

6.1.1 Line 1: `def compute_all(self):`

- Main computation method
- Connected to ”Process Samples” button

6.1.2 Line 3-5: Data Check

```
if self.data is None:
    self.status.setText("No data loaded")
    return
```

- Checks if data has been loaded
- Prevents processing without data

6.1.3 Line 8-9: Disable Button and Reset Progress

```
8 self.process_btn.setEnabled(False)
9 self.progress_bar.setValue(0)
```

- Disables button to prevent double-click
- Resets progress bar to 0

6.1.4 Line 11: Copy Data

```
11 df = self.data.copy()
```

- Creates copy of data
- Prevents modifying original

6.1.5 Line 13-16: Add Sample IDs

```
13 df["Sample"] = [
14     f"SAMPLE_{i+1:03d}"
15     for i in range(len(df))
16 ]
```

- Creates new column "Sample"
- Format: SAMPLE_001, SAMPLE_002, ...
- `:03d`: Zero-padded to 3 digits
- **Documentation**: https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions

6.1.6 Line 18: Compute Indices

```
18 indices = compute_indices(df[["Ra", "Th", "K", "dRa", "dTh", "dK"]])
```

- Calls computation engine function
- Passes only required columns
- Returns DataFrame with all indices

6.1.7 Line 21-25: Round Values

```
21 for col in indices.columns:
22     if "AEDE" in col:
23         indices[col] = indices[col].round(6)
24     else:
25         indices[col] = indices[col].round(3)
```

- Loops through all columns
- AEDE columns: 6 decimal places (very small values)
- Other columns: 3 decimal places
- **Method**: `Series.round()`
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.Series.round.html

6.1.8 Line 27: Insert Sample Column

```
27 indices.insert(0, "Sample", df["Sample"])
```

- Inserts "Sample" column at position 0 (first column)
- **Method**: `DataFrame.insert()`
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.insert.html

6.1.9 Line 29-41: Safety Classification

```
29 indices["Status"] = indices.apply(
30     lambda row:
31     "Safe"
32     if (
33         row["Rad_Eq"] <= 370 and
34         row["Hex"] <= 1 and
35         row["Hin"] <= 1 and
36         row["Dose_Rate"] <= 59 and
37         row["AEDE"] <= 1
38     )
39     else "Unsafe",
40     axis=1
41 )
```

- Creates new "Status" column
- `indices.apply(..., axis=1)`: Applies to each row
- `lambda row`: Anonymous function for each row
- Checks all five thresholds

- "Safe" if ALL conditions met, "Unsafe" otherwise
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.apply.html

6.1.10 Line 43: Store Results

```
43 self.results = indices
```

- Stores computed results in instance variable
- Available for table display, plotting, export

6.1.11 Line 48-72: Table Display

```
48 self.table.setRowCount(len(indices))
49 self.table.setColumnCount(len(indices.columns))
50 self.table.setHorizontalHeaderLabels(indices.columns)
51
52 for i in range(len(indices)):
53     for j in range(len(indices.columns)):
54
55         item = QTableWidgetItem(str(indices.iloc[i, j]))
56
57         if indices.columns[j] == "Status":
58             if indices.iloc[i, j] == "Safe":
59                 item.setBackground(Qt.green)
60             else:
61                 item.setBackground(Qt.red)
62
63         self.table.setItem(i, j, item)
```

- **Line 48-50**: Sets table dimensions and headers
- **Line 52-65**: Nested loops fill each cell
- **Line 54**: Converts value to string for display
- **Line 56-61**: Color-codes Status column
  - Safe → Green background
  - Unsafe → Red background
- **Line 63**: Places item in table
- **Documentation**: https://doc.qt.io/qt-5/qtablewidget.html

6.1.12 Line 68-73: Enable Buttons

```
68 self.export_btn.setEnabled(True)
69 self.save_plot_btn.setEnabled(True)
70 self.plot_btn.setEnabled(True)
71 self.process_btn.setEnabled(True)
72 self.progress_bar.setValue(100)
```

- Enables all export/plot buttons
- Re-enables process button
- Sets progress bar to 100

6.1.13 Line 75-81: Show Summary

```
75 safe_count = (indices["Status"] == "Safe").sum()
76 unsafe_count = (indices["Status"] == "Unsafe").sum()
77
78 self.status.setText(
79     f"    Processing complete! {safe_count} safe, {unsafe_count} unsafe
       samples"
80 )
81 self.status.setStyleSheet("color: lightgreen;")
```

- Counts Safe and Unsafe samples
- Shows summary in status bar
- Green color for success

6.1.14 Line 84: Switch to Table Tab

```
84 self.tabs.setCurrentIndex(1)
```

- Switches to table tab (index 1)
- User sees results immediately
- `setCurrentIndex()` method of QTabWidget
- **Documentation**: https://doc.qt.io/qt-5/qtabwidget.html#currentIndex-prop

6.1.15 Line 87: Generate Initial Plot

```
87 self.update_plot()
```

- Creates initial correlation plot
- Uses current view selection
- User can switch to plot tab later

6.1.16 Line 89-93: Error Handling

```
89 except Exception as e:
90     self.status.setText("    Computation Error")
91     self.status.setStyleSheet("color: red;")
92     self.process_btn.setEnabled(True)
93     print(e)
```

- Catches any computation error
- Shows red error message
- Re-enables process button for retry
- Prints error details to console

7 Plot Update Method

```
def update_plot(self):
    """Update the plot based on selected view"""

    if self.data is None or self.results is None:
        self.status.setText("No data to plot")
        return

    view = self.plot_view.currentText()

    if view == "Combined (Ra + Th + K)":
        self.plot_combined()
    elif view == "Ra Only":
        self.plot_single("Ra", "blue", "Ra-226")
    elif view == "Th Only":
        self.plot_single("Th", "red", "Th-232")
    elif view == "K Only":
        self.plot_single("K", "green", "K-40")

    self.canvas.draw()
    self.status.setText(f"    Plot updated: {view}")
    self.status.setStyleSheet("color: cyan;")
```

Listing 14: Plot Update Method

7.1 Line-by-Line Explanation

7.1.1 Line 1: `def update_plot(self):`

- Main plot update method
- Called by dropdown selection AND Generate Plot button

7.1.2 Line 4-6: Data Check

```
if self.data is None or self.results is None:
    self.status.setText("No data to plot")
    return
```

- Checks if data and results exist
- Prevents plotting without data

7.1.3 Line 8: Get Current View

```
view = self.plot_view.currentText()
```

- Gets selected text from dropdown
- `currentText()` method of QComboBox
- **Documentation**: https://doc.qt.io/qt-5/qcombobox.html#currentText-prop

7.1.4 Line 10-16: Select Plot Type

```
10 if view == "Combined (Ra + Th + K)":
11     self.plot_combined()
12 elif view == "Ra Only":
13     self.plot_single("Ra", "blue", "Ra-226")
14 elif view == "Th Only":
15     self.plot_single("Th", "red", "Th-232")
16 elif view == "K Only":
17     self.plot_single("K", "green", "K-40")
```

- Branching based on dropdown selection
- Combined view: all three nuclides
- Single views: one nuclide with specific color

7.1.5 Line 18-20: Redraw and Update Status

```
18 self.canvas.draw()
19 self.status.setText(f"    Plot updated: {view}")
20 self.status.setStyleSheet("color: cyan;")
```

- `canvas.draw()`: Redraws the canvas (critical!)
- Shows success message with current view
- Cyan color for plot updates

8 Combined Plot Method

```
def plot_combined(self):
    """Plot all three nuclides on one graph"""

    self.figure.clear()
    ax = self.figure.add_subplot(111)

    # Extract data
    Ra = self.data["Ra"].values
    Th = self.data["Th"].values
    K = self.data["K"].values

    dRa = self.data["dRa"].values
    dTh = self.data["dTh"].values
    dK = self.data["dK"].values

    Dose = self.results["Dose_Rate"].values
    dDose = self.results["dDose_Rate"].values

    # Sort for clean plotting
    ra_idx = np.argsort(Ra)
    th_idx = np.argsort(Th)
    k_idx = np.argsort(K)

    # ---- Ra ----
    ax.errorbar(
        Ra[ra_idx],
        Dose[ra_idx],
        xerr=dRa[ra_idx],
        yerr=dDose[ra_idx],
        fmt='o',
        color='blue',
        ecolor='lightblue',
        elinewidth=1,
        capsize=3,
        label='Ra-226'
    )
    ra_fit = np.polyfit(Ra, Dose, 1)
    ra_line = np.poly1d(ra_fit)
    ax.plot(Ra[ra_idx], ra_line(Ra[ra_idx]), "--", color="blue",
    linewidth=2)

    # ---- Th ----
    ax.errorbar(
        Th[th_idx],
        Dose[th_idx],
        xerr=dTh[th_idx],
        yerr=dDose[th_idx],
        fmt='s',
        color='red',
        ecolor='pink',
        elinewidth=1,
        capsize=3,
        label='Th-232'
    )
    th_fit = np.polyfit(Th, Dose, 1)
```

```
    th_line = np.poly1d(th_fit)
    ax.plot(Th[th_idx], th_line(Th[th_idx]), "--", color="red", linewidth=2)

    # ---- K ----
    ax.errorbar(
        K[k_idx],
        Dose[k_idx],
        xerr=dK[k_idx],
        yerr=dDose[k_idx],
        fmt='^',
        color='green',
        ecolor='lightgreen',
        elinewidth=1,
        capsize=3,
        label='K-40'
    )
    k_fit = np.polyfit(K, Dose, 1)
    k_line = np.poly1d(k_fit)
    ax.plot(K[k_idx], k_line(K[k_idx]), "--", color="green", linewidth=2)

    # ---- R  Values ----
    ra_r2 = np.corrcoef(Ra, Dose)[0,1]**2
    th_r2 = np.corrcoef(Th, Dose)[0,1]**2
    k_r2 = np.corrcoef(K, Dose)[0,1]**2

    ax.text(0.05, 0.95, f'Ra-226: R  = {ra_r2:.3f}', transform=ax.transAxes,
            verticalalignment='top', bbox=dict(boxstyle='round', facecolor='white', alpha=0.8),
            color='blue')
    ax.text(0.05, 0.90, f'Th-232: R  = {th_r2:.3f}', transform=ax.transAxes,
            verticalalignment='top', bbox=dict(boxstyle='round', facecolor='white', alpha=0.8),
            color='red')
    ax.text(0.05, 0.85, f'K-40: R  = {k_r2:.3f}', transform=ax.transAxes,
            verticalalignment='top', bbox=dict(boxstyle='round', facecolor='white', alpha=0.8),
            color='green')

    # ---- Formatting ----
    ax.set_ylim(bottom=0, top=max(Dose) * 1.1)
    ax.set_xlim(left=0)

    ax.set_title("Dose Rate Correlation with All Radionuclides", fontsize=12, fontweight='bold')
    ax.set_xlabel("Radionuclide Concentration (Bq/kg)", fontsize=11)
    ax.set_ylabel("Dose Rate (nGy/h)", fontsize=11)
    ax.legend(loc='lower right')
    ax.grid(True, alpha=0.3)

    # ---- Statistics Box ----
    stats_text = f"""
    Total Samples: {len(Dose)}
    Mean Dose Rate: {np.mean(Dose):.2f}    {np.std(Dose):.2f} nGy/h
```

```
    Dose Rate Range: {np.min(Dose):.2f} - {np.max(Dose):.2f} nGy/h
    """
    ax.text(0.97, 0.03, stats_text, transform=ax.transAxes,
            fontsize=9, verticalalignment='bottom', horizontalalignment='right',
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.85))

    self.figure.tight_layout()
```

Listing 15: Combined Plot Method

8.1 Line-by-Line Explanation

8.1.1 Line 1-3: Function Definition

```
def plot_combined(self):
    """Plot all three nuclides on one graph"""
```

- Defines combined plot function
- Called by `update_plot()` for combined view

8.1.2 Line 5-6: Clear and Create Axes

```
self.figure.clear()
ax = self.figure.add_subplot(111)
```

- `clear()`: Removes existing plot
- `add_subplot(111)`: Creates single subplot
- **Documentation**: https://matplotlib.org/stable/api/figure_api.html#matplotlib.figure.Figure.add_subplot

8.1.3 Line 9-19: Extract Data

```
Ra = self.data["Ra"].values
Th = self.data["Th"].values
K = self.data["K"].values

dRa = self.data["dRa"].values
dTh = self.data["dTh"].values
dK = self.data["dK"].values

Dose = self.results["Dose_Rate"].values
dDose = self.results["dDose_Rate"].values
```

- Extracts data from stored DataFrames
- `.values`: Converts to NumPy arrays (faster)
- Separates values and uncertainties

8.1.4 Line 22-24: Sort for Clean Plotting

```
22 ra_idx = np.argsort(Ra)
23 th_idx = np.argsort(Th)
24 k_idx = np.argsort(K)
```

- `np.argsort()`: Returns indices that sort the array
- Ensures regression lines plot smoothly
- **Documentation**: https://numpy.org/doc/stable/reference/generated/numpy.argsort.html

8.1.5 Line 27-41: Plot Ra-226

```
27 ax.errorbar(
28     Ra[ra_idx],
29     Dose[ra_idx],
30     xerr=dRa[ra_idx],
31     yerr=dDose[ra_idx],
32     fmt='o',
33     color='blue',
34     ecolor='lightblue',
35     elinewidth=1,
36     capsize=3,
37     label='Ra-226'
38 )
```

- **Method**: `ax.errorbar()`
- **Documentation**: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.errorbar.html
- **Parameters**:
  - `x`, `y`: Data points (sorted)
  - `xerr`, `yerr`: Error bars
  - `fmt='o'`: Circle marker
  - `color='blue'`: Blue points
  - `ecolor='lightblue'`: Light blue error bars
  - `elinewidth=1`: Thin error bars
  - `capsize=3`: Caps at ends
  - `label='Ra-226'`: Legend label

8.1.6 Line 42-44: Ra Regression

```
42 ra_fit = np.polyfit(Ra, Dose, 1)
43 ra_line = np.poly1d(ra_fit)
44 ax.plot(Ra[ra_idx], ra_line(Ra[ra_idx]), "--", color="blue", linewidth
   =2)
```

- `np.polyfit()`: Fits straight line (degree 1)
- `np.poly1d()`: Creates polynomial function
- `ax.plot()`: Draws dashed line
- **Documentation**: https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html

8.1.7 Line 47-61: Plot Th-232

- Same structure as Ra
- `fmt='s'`: Square marker (distinct from Ra)
- `color='red'`: Red points
- `ecolor='pink'`: Pink error bars

8.1.8 Line 64-78: Plot K-40

- Same structure as Ra
- `fmt='`$'$: $Trianglemarker(distinct)$
- `color='green'`: Green points
- `ecolor='lightgreen'`: Light green error bars

8.1.9 Line 81-83: Calculate R^2 Values

```
81 ra_r2 = np.corrcoef(Ra, Dose)[0,1]**2
82 th_r2 = np.corrcoef(Th, Dose)[0,1]**2
83 k_r2 = np.corrcoef(K, Dose)[0,1]**2
```

- `np.corrcoef()`: Returns correlation matrix
- `[0,1]`: Extracts correlation coefficient
- `**2`: Squares to get R^2
- **Documentation**: https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html

8.1.10 Line 85-94: Display R^2 Values

```
85 ax.text(0.05, 0.95, f'Ra-226: R  = {ra_r2:.3f}', transform=ax.
   transAxes,
86         verticalalignment='top', bbox=dict(boxstyle='round', facecolor=
   'white', alpha=0.8),
87         color='blue')
```

- **Method**: `ax.text()`
- **Documentation**: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.text.html
- `0.05, 0.95`: Position (top-left)
- `transform=ax.transAxes`: Relative coordinates (0-1)
- `bbox`: White rounded box with 80
- Color matches nuclide color

8.1.11 Line 97-105: Formatting

```
97 ax.set_ylim(bottom=0, top=max(Dose) * 1.1)
98 ax.set_xlim(left=0)
99
100 ax.set_title("Dose Rate Correlation with All Radionuclides", fontsize
   =12, fontweight='bold')
101 ax.set_xlabel("Radionuclide Concentration (Bq/kg)", fontsize=11)
102 ax.set_ylabel("Dose Rate (nGy/h)", fontsize=11)
103 ax.legend(loc='lower right')
104 ax.grid(True, alpha=0.3)
```

- `set_ylim()`: Y-axis from 0 to 10
- `set_xlim(left=0)`: X-axis starts at 0
- `set_title()`: Bold title
- `set_xlabel()`, `set_ylabel()`: Axis labels
- `legend(loc='lower right')`: Legend position
- `grid(True, alpha=0.3)`: Subtle grid

8.1.12 Line 108-117: Statistics Box

```
108 stats_text = f"""
109 Total Samples: {len(Dose)}
110 Mean Dose Rate: {np.mean(Dose):.2f}    {np.std(Dose):.2f} nGy/h
111 Dose Rate Range: {np.min(Dose):.2f} - {np.max(Dose):.2f} nGy/h
112 """
113 ax.text(0.97, 0.03, stats_text, transform=ax.transAxes,
114         fontsize=9, verticalalignment='bottom', horizontalalignment='
   right',
115         bbox=dict(boxstyle='round', facecolor='white', alpha=0.85))
```

- Shows summary statistics on plot
- Bottom-right corner (0.97, 0.03)
- Includes: sample count, mean ± std, range
- White rounded box for readability

8.1.13 Line 119: Tight Layout

```
119 self.figure.tight_layout()
```

- `tight_layout()`: Adjusts spacing automatically
- Prevents overlapping labels
- **Documentation**: https://matplotlib.org/stable/api/figure_api.html#matplotlib.figure.Figure.tight_layout

9 Single Plot Method

```
def plot_single(self, nuclide, color, label):
    """Plot dose rate vs a single radionuclide"""

    self.figure.clear()
    ax = self.figure.add_subplot(111)

    # Get data for specific nuclide
    if nuclide == "Ra":
        x = self.data["Ra"].values
        dx = self.data["dRa"].values
        x_label = "Ra-226 Concentration (Bq/kg)"
    elif nuclide == "Th":
        x = self.data["Th"].values
        dx = self.data["dTh"].values
        x_label = "Th-232 Concentration (Bq/kg)"
    else:  # K
        x = self.data["K"].values
        dx = self.data["dK"].values
        x_label = "K-40 Concentration (Bq/kg)"

    dose = self.results["Dose_Rate"].values
    ddose = self.results["dDose_Rate"].values

    # Sort for clean plotting
    idx = np.argsort(x)
    x_sorted = x[idx]
    dose_sorted = dose[idx]
    dx_sorted = dx[idx]
    ddose_sorted = ddose[idx]

    # Plot with error bars
    ax.errorbar(x_sorted, dose_sorted,
                xerr=dx_sorted, yerr=ddose_sorted,
                fmt='o', color=color, ecolor='lightblue',
                elinewidth=1, capsize=3, label=label)

    # Regression line
    fit = np.polyfit(x, dose, 1)
    line = np.poly1d(fit)
    ax.plot(x_sorted, line(x_sorted), '--', color=color, linewidth=2)

    # R  value
    r2 = np.corrcoef(x, dose)[0,1]**2
    ax.text(0.05, 0.95, f'R  = {r2:.3f}', transform=ax.transAxes,
            verticalalignment='top', bbox=dict(boxstyle='round',
    facecolor='white', alpha=0.8))

    # Formatting
    ax.set_ylim(bottom=0, top=max(dose) * 1.1)
    ax.set_xlim(left=0)

    ax.set_title(f'Dose Rate vs {label} Concentration', fontsize=12,
    fontweight='bold')
    ax.set_xlabel(x_label, fontsize=11)
    ax.set_ylabel('Dose Rate (nGy/h)', fontsize=11)
```

```
    ax.legend(loc='lower right')
    ax.grid(True, alpha=0.3)

    # Statistics box
    stats_text = f"""
    Samples: {len(dose)}
    Mean {label}: {np.mean(x):.2f}    {np.std(x):.2f} Bq/kg
    Mean Dose: {np.mean(dose):.2f}    {np.std(dose):.2f} nGy/h
    Correlation: R   = {r2:.3f}
    """
    ax.text(0.97, 0.03, stats_text, transform=ax.transAxes,
            fontsize=9, verticalalignment='bottom', horizontalalignment='right',
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.85))

    self.figure.tight_layout()
```

Listing 16: Single Plot Method

9.1 Line-by-Line Explanation

9.1.1 Line 1-2: Function Definition

```
def plot_single(self, nuclide, color, label):
    """Plot dose rate vs a single radionuclide"""
```

- **Parameters**:
  - `nuclide`: "Ra", "Th", or "K"
  - `color`: Color for the plot
  - `label`: Full name for display
- Called by `update_plot()` for single views

9.1.2 Line 8-19: Select Nuclide Data

```
if nuclide == "Ra":
    x = self.data["Ra"].values
    dx = self.data["dRa"].values
    x_label = "Ra-226 Concentration (Bq/kg)"
elif nuclide == "Th":
    x = self.data["Th"].values
    dx = self.data["dTh"].values
    x_label = "Th-232 Concentration (Bq/kg)"
else:  # K
    x = self.data["K"].values
    dx = self.data["dK"].values
    x_label = "K-40 Concentration (Bq/kg)"
```

- Branching based on nuclide parameter
- Sets `x` (concentration) and `dx` (uncertainty)
- Sets appropriate axis label

9.1.3 Line 21-22: Get Dose Data

```
21 dose = self.results["Dose_Rate"].values
22 ddose = self.results["dDose_Rate"].values
```

- Same dose data for all nuclides
- Used as y-values in all plots

9.1.4 Line 25-30: Sort for Plotting

```
25 idx = np.argsort(x)
26 x_sorted = x[idx]
27 dose_sorted = dose[idx]
28 dx_sorted = dx[idx]
29 ddose_sorted = ddose[idx]
```

- Sorts all data by concentration
- Ensures smooth regression line
- All arrays aligned by sorted order

9.1.5 Line 33-37: Plot with Error Bars

```
33 ax.errorbar(x_sorted, dose_sorted,
34              xerr=dx_sorted, yerr=ddose_sorted,
35              fmt='o', color=color, ecolor='lightblue',
36              elinewidth=1, capsize=3, label=label)
```

- All nuclides use circle markers
- Color parameter determines point color
- Error bars in light blue
- Label from parameter

9.1.6 Line 40-42: Regression Line

```
40 fit = np.polyfit(x, dose, 1)
41 line = np.poly1d(fit)
42 ax.plot(x_sorted, line(x_sorted), '--', color=color, linewidth=2)
```

- Same regression method as combined plot
- Color matches nuclide

9.1.7 Line 45-48: R^2 Display

```
r2 = np.corrcoef(x, dose)[0,1]**2
ax.text(0.05, 0.95, f'R  = {r2:.3f}', transform=ax.transAxes,
        verticalalignment='top', bbox=dict(boxstyle='round', facecolor=
    'white', alpha=0.8))
```

- Single R^2 value (only one nuclide)
- Positioned at top-left

9.1.8 Line 51-58: Formatting

```
ax.set_ylim(bottom=0, top=max(dose) * 1.1)
ax.set_xlim(left=0)

ax.set_title(f'Dose Rate vs {label} Concentration', fontsize=12,
    fontweight='bold')
ax.set_xlabel(x_label, fontsize=11)
ax.set_ylabel('Dose Rate (nGy/h)', fontsize=11)
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3)
```

- Dynamic title using label parameter
- Dynamic x-axis label
- Same formatting as combined plot

9.1.9 Line 61-70: Statistics Box

```
stats_text = f"""
Samples: {len(dose)}
Mean {label}: {np.mean(x):.2f}    {np.std(x):.2f} Bq/kg
Mean Dose: {np.mean(dose):.2f}    {np.std(dose):.2f} nGy/h
Correlation: R  = {r2:.3f}
"""
ax.text(0.97, 0.03, stats_text, transform=ax.transAxes,
        fontsize=9, verticalalignment='bottom', horizontalalignment='
    right',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.85))
```

- Includes nuclide-specific statistics
- Mean concentration with standard deviation
- Mean dose with standard deviation
- Correlation strength

9.1.10 Line 72: Tight Layout

```
self.figure.tight_layout()
```

- Same as combined plot

10 Save Plot Method

```
def save_plot(self):

    if self.figure is None:
        self.status.setText("No plot available")
        return

    path, _ = QFileDialog.getSaveFileName(
        self,
        "Save Plot",
        "",
        "PNG (*.png);;PDF (*.pdf);;SVG (*.svg)"
    )

    if path:
        self.figure.savefig(
            path,
            dpi=300,
            bbox_inches="tight"
        )

        self.status.setText("    Plot saved successfully")
        self.status.setStyleSheet("color: lightgreen;")
```

Listing 17: Save Plot Method

10.1 Line-by-Line Explanation

10.1.1 Line 1: `def save_plot(self):`

- Saves current plot to file
- Connected to "Save Plot" button

10.1.2 Line 3-5: Check Plot Exists

```
if self.figure is None:
    self.status.setText("No plot available")
    return
```

- Prevents saving when no plot exists

10.1.3 Line 7-13: File Dialog

```
path, _ = QFileDialog.getSaveFileName(
    self,
    "Save Plot",
    "",
    "PNG (*.png);;PDF (*.pdf);;SVG (*.svg)"
)
```

- **Method**: `QFileDialog.getSaveFileName()`

- **Documentation**: https://doc.qt.io/qt-5/qfiledialog.html#getSaveFileName
- Multiple format options: PNG, PDF, SVG

10.1.4 Line 15-20: Save Figure

```
15 if path:
16     self.figure.savefig(
17         path,
18         dpi=300,
19         bbox_inches="tight"
20     )
```

- `Figure.savefig()`: Saves figure to file
- `dpi=300`: Print quality (300 dots per inch)
- `bbox_inches="tight"`: Removes extra whitespace
- **Documentation**: https://matplotlib.org/stable/api/figure_api.html#matplotlib.figure.Figure.savefig

10.1.5 Line 22-23: Update Status

```
22 self.status.setText("    Plot saved successfully")
23 self.status.setStyleSheet("color: lightgreen;")
```

- Success message in green
- User knows save completed

11 Save Results Method

```
def save_results(self):

    if self.results is None:
        self.status.setText("Nothing to save")
        return

    path, _ = QFileDialog.getSaveFileName(
        self,
        "Save Results",
        "",
        "CSV Files (*.csv)"
    )

    if path:
        self.results.to_csv(path, index=False)
        self.status.setText("    Results exported")
        self.status.setStyleSheet("color: lightgreen;")
```

Listing 18: Save Results Method

11.1 Line-by-Line Explanation

11.1.1 Line 1: `def save_results(self):`

- Exports results to CSV
- Connected to "Export CSV" button

11.1.2 Line 3-5: Check Results Exist

```
if self.results is None:
    self.status.setText("Nothing to save")
    return
```

- Prevents saving when no results exist

11.1.3 Line 7-11: File Dialog

```
path, _ = QFileDialog.getSaveFileName(
    self,
    "Save Results",
    "",
    "CSV Files (*.csv)"
)
```

- CSV format only
- Universal format for data export

11.1.4 Line 13-16: Export to CSV

```
13 if path:
14     self.results.to_csv(path, index=False)
15     self.status.setText("    Results exported")
16     self.status.setStyleSheet("color: lightgreen;")
```

- `to_csv()`: Exports DataFrame to CSV
- `index=False`: Excludes row numbers
- **Documentation**: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html

12 Style Application Method

```
def apply_style(self):

    self.setStyleSheet("""
        QWidget {
            background-color: #1e1e2f;
            color: white;
            font-size: 14px;
        }

        QLabel#title {
            font-size: 24px;
            font-weight: bold;
            color: #00d4ff;
            padding: 10px;
        }

        QLabel#status {
            padding: 10px;
            background-color: #2b2b3d;
            border-radius: 6px;
            font-weight: bold;
        }

        QGroupBox {
            font-weight: bold;
            border: 2px solid #444;
            border-radius: 8px;
            margin-top: 10px;
            padding-top: 10px;
        }

        QGroupBox::title {
            subcontrol-origin: margin;
            left: 10px;
            padding: 0 5px 0 5px;
            color: #00d4ff;
        }

        QPushButton {
            background-color: #2d89ef;
            border: none;
            border-radius: 8px;
            padding: 10px;
            font-weight: bold;
            color: white;
        }

        QPushButton:hover {
            background-color: #45a1ff;
        }

        QPushButton:disabled {
            background-color: #444;
            color: #888;
        }
```

```
56
57          QComboBox {
58              padding: 8px;
59              border-radius: 6px;
60              background-color: #2b2b3d;
61              border: 2px solid #555;
62              color: white;
63              min-width: 150px;
64          }
65
66          QComboBox::drop-down {
67              border: none;
68          }
69
70          QComboBox::down-arrow {
71              image: none;
72              border-left: 5px solid transparent;
73              border-right: 5px solid transparent;
74              border-top: 5px solid white;
75              margin-right: 5px;
76          }
77
78          QTableWidget {
79              background-color: #2b2b3d;
80              gridline-color: #444;
81              alternate-background-color: #333344;
82          }
83
84          QTableWidget::item {
85              padding: 5px;
86          }
87
88          QHeaderView::section {
89              background-color: #444;
90              padding: 8px;
91              font-weight: bold;
92              border: 1px solid #555;
93          }
94
95          QProgressBar {
96              border: 2px solid #444;
97              border-radius: 6px;
98              text-align: center;
99              background-color: #2b2b3d;
100         }
101
102         QProgressBar::chunk {
103             background-color: #2d89ef;
104             border-radius: 4px;
105         }
106
107         QTabWidget::pane {
108             border: 1px solid #444;
109             border-radius: 8px;
110             background-color: #1e1e2f;
111         }
112
113         QTabBar::tab {
```

```
            background-color: #333;
            padding: 10px 20px;
            margin: 2px;
            border-radius: 6px;
        }

        QTabBar::tab:selected {
            background-color: #2d89ef;
        }

        QTabBar::tab:hover:!selected {
            background-color: #444;
        }

        QLabel {
            color: white;
        }
    """)
```

Listing 19: Style Application Method

12.1 Line-by-Line Explanation

12.1.1 Line 1: `def apply_style(self):`

- Applies CSS-like styles to the application
- Called at the end of $_{init}$
- `Uses Qt Style Sheets (QSS)`
- `Documentation:` https://doc.qt.io/qt-5/stylesheet.html

12.1.2 Line 3-7: Global Styling

```
QWidget {
    background-color: #1e1e2f;
    color: white;
    font-size: 14px;
}
```

- Applies to ALL widgets
- Dark background (1e1e2f)
- White text
- 14px base font size

12.1.3 Line 9-14: Title Styling

```
QLabel#title {
    font-size: 24px;
    font-weight: bold;
    color: #00d4ff;
    padding: 10px;
}
```

- `QLabel#title`: Only label with objectName="title"
- Large (24px), bold, cyan text
- 10px padding

12.1.4 Line 16-21: Status Styling

```
QLabel#status {
    padding: 10px;
    background-color: #2b2b3d;
    border-radius: 6px;
    font-weight: bold;
}
```

- `QLabel#status`: Only status label
- Rounded corners (6px)
- Slightly lighter background
- Bold text

12.1.5 Line 23-32: Group Box Styling

```
QGroupBox {
    font-weight: bold;
    border: 2px solid #444;
    border-radius: 8px;
    margin-top: 10px;
    padding-top: 10px;
}

QGroupBox::title {
    subcontrol-origin: margin;
    left: 10px;
    padding: 0 5px 0 5px;
    color: #00d4ff;
}
```

- `QGroupBox`: Bordered container
- Gray border (2px), rounded corners (8px)
- `QGroupBox::title`: Title text
- Cyan color, positioned 10px from left

12.1.6 Line 34-47: Button Styling

```
34 QPushButton {
35     background-color: #2d89ef;
36     border: none;
37     border-radius: 8px;
38     padding: 10px;
39     font-weight: bold;
40     color: white;
41 }
42
43 QPushButton:hover {
44     background-color: #45a1ff;
45 }
46
47 QPushButton:disabled {
48     background-color: #444;
49     color: #888;
50 }
```

- `QPushButton`: Blue background, white text
- `::hover`: Lighter blue when mouse over
- `::disabled`: Gray when disabled

12.1.7 Line 49-68: ComboBox Styling

```
49 QComboBox {
50     padding: 8px;
51     border-radius: 6px;
52     background-color: #2b2b3d;
53     border: 2px solid #555;
54     color: white;
55     min-width: 150px;
56 }
57
58 QComboBox::drop-down {
59     border: none;
60 }
61
62 QComboBox::down-arrow {
63     image: none;
64     border-left: 5px solid transparent;
65     border-right: 5px solid transparent;
66     border-top: 5px solid white;
67     margin-right: 5px;
68 }
```

- Dark dropdown with gray border
- Custom triangle arrow (created with CSS borders)
- `border-left/right: transparent` + `border-top: white` = downward triangle

12.1.8 Line 70-82: Table Styling

```
70 QTableWidget {
71     background-color: #2b2b3d;
72     gridline-color: #444;
73     alternate-background-color: #333344;
74 }
75
76 QTableWidget::item {
77     padding: 5px;
78 }
79
80 QHeaderView::section {
81     background-color: #444;
82     padding: 8px;
83     font-weight: bold;
84     border: 1px solid #555;
85 }
```

- Dark table with subtle grid lines
- `alternate-background-color`: Zebra striping
- Cells: 5px padding
- Headers: Bold, 8px padding, gray background

12.1.9 Line 84-92: Progress Bar Styling

```
84 QProgressBar {
85     border: 2px solid #444;
86     border-radius: 6px;
87     text-align: center;
88     background-color: #2b2b3d;
89 }
90
91 QProgressBar::chunk {
92     background-color: #2d89ef;
93     border-radius: 4px;
94 }
```

- Gray border, rounded corners
- Blue fill (matches buttons)
- Percentage text centered

12.1.10 Line 94-112: Tab Styling

```
94 QTabWidget::pane {
95     border: 1px solid #444;
96     border-radius: 8px;
97     background-color: #1e1e2f;
98 }
99
```

```
100 QTabBar::tab {
101     background-color: #333;
102     padding: 10px 20px;
103     margin: 2px;
104     border-radius: 6px;
105 }
106
107 QTabBar::tab:selected {
108     background-color: #2d89ef;
109 }
110
111 QTabBar::tab:hover:!selected {
112     background-color: #444;
113 }
```

- `QTabWidget::pane`: Content area with rounded corners
- `QTabBar::tab`: Each tab with padding and rounded corners
- `::tab:selected`: Blue for active tab
- `::tab:hover:!selected`: Slightly lighter on hover

12.1.11 Line 114-116: Label Styling

```
114 QLabel {
115     color: white;
116 }
```

- Ensures ALL labels have white text
- Overrides any default dark text

13 Application Entry Point

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    app.setStyle('Fusion')
    window = MainApp()
    window.show()
    sys.exit(app.exec_())
```

Listing 20: Application Entry Point

13.1 Line-by-Line Explanation

13.1.1 Line 1: `if __name__ == "__main__":`

- **Documentation**: https://docs.python.org/3/library/__main__.html
- Only runs when script is executed directly
- Prevents code from running when imported

13.1.2 Line 2: `app = QApplication(sys.argv)`

- **Method**: `QApplication()`
- **Documentation**: https://doc.qt.io/qt-5/qapplication.html
- Creates the Qt application object
- **Required**: Exactly ONE per application
- `sys.argv`: Passes command-line arguments

13.1.3 Line 3: `app.setStyle('Fusion')`

- **Method**: `QApplication.setStyle()`
- **Documentation**: https://doc.qt.io/qt-5/qapplication.html#setStyle
- Sets Fusion style (modern, cross-platform)
- Consistent look across Windows, Mac, Linux

13.1.4 Line 4: `window = MainApp()`

- Creates instance of MainApp
- Runs `__init__` method
- Builds entire UI
- Window exists but is hidden

13.1.5 Line 5: `window.show()`

- **Method**: `QWidget.show()`
- **Documentation**: https://doc.qt.io/qt-5/qwidget.html#show
- Makes window visible on screen
- Without this, window would be invisible

13.1.6 Line 6: `sys.exit(app.exec_())`

- **Method**: `QApplication.exec()`
- **Documentation**: https://doc.qt.io/qt-5/qapplication.html#exec
- Starts the Qt event loop
- Blocks until window is closed
- `sys.exit()`: Passes exit code to system
- **Why underscore**: Avoids conflict with Python's `exec`

14 Summary of Key Documentation Links

14.1 Python Standard Library

- `sys` module: https://docs.python.org/3/library/sys.html
- `__name__` variable: https://docs.python.org/3/library/__main__.html

14.2 NumPy Documentation

- `np.sqrt()`: https://numpy.org/doc/stable/reference/generated/numpy.sqrt.html
- `np.argsort()`: https://numpy.org/doc/stable/reference/generated/numpy.argsort.html
- `np.polyfit()`: https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html
- `np.poly1d()`: https://numpy.org/doc/stable/reference/generated/numpy.poly1d.html
- `np.corrcoef()`: https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html

14.3 pandas Documentation

- `pd.read_csv()`: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
- `pd.DataFrame()`: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html
- `DataFrame.to_csv()`: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html
- `DataFrame.apply()`: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.apply.html
- `DataFrame.get()`: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.get.html
- `Series.round()`: https://pandas.pydata.org/docs/reference/api/pandas.Series.round.html

14.4 Matplotlib Documentation

- `Figure`: https://matplotlib.org/stable/api/figure_api.html
- `FigureCanvasQTAgg`: https://matplotlib.org/stable/api/backend_qt5agg_api.html

- `Axes.errorbar()`: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.errorbar.html
- `Axes.text()`: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.text.html
- `Figure.savefig()`: https://matplotlib.org/stable/api/figure_api.html#matplotlib.figure.Figure.savefig

14.5 PyQt5 Documentation

- `QApplication`: https://doc.qt.io/qt-5/qapplication.html
- `QWidget`: https://doc.qt.io/qt-5/qwidget.html
- `QLabel`: https://doc.qt.io/qt-5/qlabel.html
- `QPushButton`: https://doc.qt.io/qt-5/qpushbutton.html
- `QGroupBox`: https://doc.qt.io/qt-5/qgroupbox.html
- `QComboBox`: https://doc.qt.io/qt-5/qcombobox.html
- `QTableWidget`: https://doc.qt.io/qt-5/qtablewidget.html
- `QTabWidget`: https://doc.qt.io/qt-5/qtabwidget.html
- `QProgressBar`: https://doc.qt.io/qt-5/qprogressbar.html
- `QFileDialog`: https://doc.qt.io/qt-5/qfiledialog.html
- `QHBoxLayout`: https://doc.qt.io/qt-5/qhboxlayout.html
- `QVBoxLayout`: https://doc.qt.io/qt-5/qvboxlayout.html
- `Qt Style Sheets`: https://doc.qt.io/qt-5/stylesheet.html